Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет «Высшая школа экономики»
Факультет компьютерных наук, Департамент программной инженерии

Отчёт по индивидуальному заданию № 2

по дисциплине «Архитектура компьютера и операционные системы»

Вариант 24: «Танковые бои». Многопоточная версия

Выполнил: Адаменко Мирослав Александрович
Группа: БПИ-252
Вариант: 24

Москва, 2026

Оглавление

1. Условие задачи . . . 3
2. Сценарий отображения сущностей в потоки . . . 4
   2.1. Анализ реализуемости потоками . . . 4
   2.2. Таблица «сущность → поток → примитивы» . . . 4
   2.3. Фазы такта . . . 6
3. Синхронизация . . . 6
   3.1. tanks_sem.c: семафоры и мьютексы . . . 6
   3.2. tanks_cond.c: мьютексы, условные переменные и барьер . . . 7
   3.3. Инварианты и детерминизм . . . 9
4. Интерфейс . . . 10
5. Запуск и завершение . . . 11
6. Тесты и результаты . . . 13
7. Соответствие требованиям . . . 14
   7.1. Минимальные требования . . . 14
   7.2. Дополнительные требования . . . 15
8. Ограничения . . . 15

1. Условие задачи

На клеточном поле действуют танковые подразделения двух сторон. Танки перемещаются, наблюдают доступные цели и обстреливают противника. На каждом такте танк может изменить позицию, выбрать цель и сформировать выстрел. Решения сторон принимаются независимо и применяются по общей фазовой схеме, чтобы порядок перебора участников не давал одной стороне скрытого преимущества.

Стрельба выполняется прямой наводкой только по видимым целям либо по навесной траектории (выбор за разработчиком). Уничтоженные танки либо остаются на поле как препятствия, либо удаляются (режим выбирает разработчик). Стратегия может быть постоянной или меняться в ходе боя.

Входные параметры: размеры поля и препятствия; число, позиции, дальность и прочность танков; порядок фаз такта; стратегии; тип стрельбы; правила столкновений; предельное число тактов.

Отображаемые события: наблюдение, выбор намерения перемещения, разрешение конфликтов клеток, смена позиции, выбор цели, выстрел, попадание, повреждение, уничтожение, состояние поля после такта.

Инварианты: два действующих танка не занимают одну клетку; конфликтующие перемещения разрешаются однозначно; прямая наводка используется только при наличии линии видимости; решения такта формируются до раскрытия результатов противника; уничтоженный танк действует далее только как препятствие, если это предусмотрено режимом.

Завершение: уничтожение одной или обеих сторон, либо предельное число тактов, либо прерывание пользователем в режиме неограниченного выполнения; формируется итоговая статистика. Координаторы сторон и каждый танк являются самостоятельными участниками модели.

Принятые решения (унаследованы из ДЗ-1): прямой огонь; препятствия блокируют движение, наблюдение и стрельбу; уничтоженные танки удаляются с поля; танк занимает 11 × 5 символов, перемещение посимвольное; 4 модели (Maus, FV215b183, T62A, Leopard 1); приоритет при конфликте перемещения определяется массой; фазы такта: решение, выстрелы, определение уничтоженных, прицеливание и перемещение, отрисовка.

Отличие от ДЗ-1: там симуляция была последовательной, а координаторы не реализовывались. В ИДЗ-2 каждый танк, координатор команды и арбитр поля являются потоками. Механика боя ДЗ-1 сохранена, но есть отличия:

- координатор команды по умолчанию назначает цель сосредоточенного огня (фокус), и она может изменить выбор цели танка; выбор строго как в ДЗ-1 включается параметром `--no-focus`;
- у каждого танка собственный генератор `rand_r` с начальным значением `seed + 1 + i` вместо общего `rand()`;
- `rand01` делит на `RAND_MAX + 1`, а не на `RAND_MAX`; HP при попадании обрезается снизу нулём.

2. Сценарий отображения сущностей в потоки

2.1. Анализ реализуемости потоками

Сущность выносится в поток, если у неё есть собственное поведение (цикл «получить команду фазы — выполнить — отчитаться»), состояние и время жизни. Пассивные данные потоками не являются.

Сущность	**Поток?**	**Обоснование**
Танк	да	`tank_main`: по команде арбитра наблюдает поле по четырём направлениям и докладывает на доску команды (фаза наблюдения), затем выбирает цель и намерение в `plans[i]` (фаза решения). Собственное состояние: HP, перезарядка, прицел, направление, генератор `rand_r`.
Координатор команды	да	`coord_main` (по одному на команду A и B): забирает с доски донесения танков своей команды, сводит их в разведданные `intel[team]` и назначает цель фокуса огня.
Арбитр поля	да	`arbiter_main`: раздаёт фазы такта, ждёт отчётов, затем единолично применяет выстрелы, уничтожение, перемещения с разрешением конфликтов, отрисовку и задержку, проверяет конец боя и инварианты.
Главный поток	служебный	`main`: разбор аргументов и настройка, установка обработчиков сигналов, создание примитивов и потоков (`run_battle`), `join`, итоговая статистика и код выхода.
Поле, препятствия	нет	Пассивные данные; в фазах участников только читаются, пишет их один арбитр.
Снаряд / выстрел	нет	Прямая наводка мгновенна (луч `cast_ray`), поведения во времени нет; выстрелы применяет арбитр.
Журнал событий	нет	Разделяемый ресурс (структура `Journal`) под мьютексом; отдельного потока вывода нет, сбрасывает журнал в файл арбитр.

Итого потоков: $N_A + N_B$ танков + 2 координатора + 1 арбитр (+ главный поток). Для боя по умолчанию 4 + 4 танка это 8 + 2 + 1 = 11 рабочих потоков; в журнале это строка `# потоки: танков 8, координаторов 2, арбитр 1`. Поток уничтоженного танка завершается (арбитр отправляет ему команду `CMD_QUIT`; в обеих программах), в журнале появляется строка «поток танка … остановлен».

2.2. Таблица «сущность → поток → примитивы»

Сущность	**Поток**	**Примитивы в tanks_sem.c**	**Примитивы в tanks_cond.c**
Танк	`tank_main`	`sem_wait(tank_go[i])` — ждёт команду фазы;	`dispatch.lock` + `cond_wait(go)` в цикле

		`board[team].lock` — запись донесения на доску; `journal.lock` (в `journal_submit`) — сдача событий; `sem_post(done)` — отчёт арбитру.	`while (cmd[id] == CMD_NONE)` — ждёт команду в своём ящике; `board[team].lock` — запись донесения; `journal.lock`; под `dispatch.lock --pending`, при нуле `cond_signal(done)` — отчёт арбитру.
Координатор команды	`coord_main`	`sem_wait(coord_go[team])`; `board[team].lock` — забрать и очистить доску; `journal.lock`; `sem_post(done)`.	До старта: `cond_wait(go)` на предикате `launch == LAUNCH_WAIT`. Далее `barrier_wait(coord_barrier)` — раунд «начало», `board[team].lock` — забрать и очистить доску, `journal.lock`, `barrier_wait` — раунд «конец».
Арбитр	`arbiter_main`	`sem_post(tank_go[i])`, `sem_post(coord_go[t])` — раздача фаз; `sem_wait(done)` — ожидание всех отчётов; `sem_timedwait(g_wake)` — прерываемая задержка; сброс журнала `journal_flush`.	Фазы танков: под `dispatch.lock` кладёт `cmd[i]`, `pending = N`, `cond_broadcast(go)`, ждёт `while (pending > 0) cond_wait(done)`. Фаза координации: два вызова `barrier_wait` (участник из трёх). Задержка — `interruptible_delay` без примитива синхронизации (маска сигналов). Остановка: `CMD_QUIT` + `broadcast`, `coord_quit` + раунд барьера. Сброс журнала `journal_flush`.
Общий журнал	—	`journal.lock`: защищает `staged`, `entries`, `count`, `cap`, `seq`, `lost`.	`journal.lock`: те же поля.

Мир (`World`) в обеих программах не защищён блокировкой: его пишет только арбитр и только тогда, когда остальные потоки стоят на своих ожиданиях. Выбор примитива для каждой фазы в `tanks_cond.c` обоснован в разделе 3.2.

2.3. Фазы такта

Порядок фаз определяется кодом `arbiter_main` (перечисление `Phase`: наблюдение, координация, решение, стрельба, уничтожение, перемещение, отрисовка, задержка).

Фаза	Танки	Координаторы	Арбитр
1. Наблюдение	наблюдают поле, пишут донесения на доску команды	ждут	раздаёт `tank_go`, ждёт N отчётов `done`
2. Координация	ждут	сводят донесения, публикуют `intel`, назначают фокус	раздаёт `coord_go`, ждёт 2 отчёта
3. Решение	выбирают цель и намерение, пишут `plans[i]`	ждут	раздаёт `tank_go`, ждёт N отчётов
4. Стрельба	ждут	ждут	применяет все выстрелы, вычитает HP
5. Уничтожение	погибшие получают `CMD_QUIT` и завершаются	ждут	определяет уничтоженных, удаляет их с поля
6. Перемещение	ждут	ждут	прицеливание и перемещение в очереди по убыванию массы, разрешение конфликтов, проверка инвариантов
7. Отрисовка	ждут	ждут	кадр на экран, события такта в журнал
8. Задержка	ждут	ждут	`interruptible_delay`, проверка конца боя

Таблица 1. Фазы такта и участие потоков (по `arbiter_main`; в tanks_cond.c порядок фаз тот же)

В фазах 1–3 мир только читается, а арбитр ждёт на семафоре `done` (в tanks_cond.c — на условной переменной `done` или на барьере); в фазах 4–8 мир пишет только арбитр, остальные потоки заблокированы на своих `tank_go`/`coord_go`.

3. Синхронизация

3.1. tanks_sem.c: семафоры и мьютексы

Семафоры POSIX (`sem_t`), все создаются `sem_init(..., 0, 0)` в `sync_init`, начальное значение 0:

Семафор	Кто post / wait	Назначение
`tank_go[i]` (по одному на танк)	post — арбитр (`run_tank_phase`,	Личный «турникет» танка: один `post` — одна фаза. Команда фазы

	retire_tank, stop_workers); wait — tank_main	(tank_cmd[i]) арбитр пишет до post, поток читает после wait.
coord_go[t] (по одному на команду)	post — арбитр (run_coord_phase); wait — coord_main	То же для координатора команды t.
done	post — каждый участник по окончании фазы; wait — арбитр столько раз, сколько раздал фаз	Счётный семафор отчётов: отчёт, пришедший раньше, не теряется.
g_wake	post — обработчик сигнала on_signal (sem_post async-signal-safe); wait (sem_timedwait) — арбитр в interruptible_delay	Прерывание задержки по SIGINT/SIGTERM без гонки «проверил флаг — уснул».

Все вызовы обёрнуты (sem_post_checked, sem_wait_checked): EINTR повторяется, иная ошибка примитива приводит к sync_fatal (сообщение, восстановление терминала, abort), потому что сломанный протокол грозит дедлоком.

Мьютексы (pthread_mutex_t), три штуки, вложенных захватов нет:

Мьютекс	Что защищает
board[0].lock, board[1].lock	Доска донесений команды (Sightings): в фазе наблюдения туда одновременно пишут все танки команды (sightings_merge), в фазе координации координатор копирует и обнуляет её.
journal.lock	Приёмник событий журнала (staged, entries, seq): в него параллельно пишут танки и координаторы (journal_submit).

Порядок захвата мьютексов задавать не нужно: поток держит не более одного мьютекса, и внутри критической секции не вызывает ничего блокирующего.

Передача управления между фазами. Арбитр записывает команду, делает sem_post на личный семафор каждого живого участника и затем sem_wait(done) ровно столько раз, сколько раздал. Цепочка happens-before: запись участника, post(done), wait(done) арбитра, затем (в следующей фазе) post(go), wait(go) участника. Благодаря ей доступ к world, plans, intel без отдельного мьютекса корректен, что подтверждено прогоном ThreadSanitizer (раздел 6).

Сигналы. SIGINT и SIGTERM заблокированы во всех потоках; разблокирует их только арбитр (pthread_sigmask), при настройке в main они открываются атомарно внутри ppoll. Остальные потоки узнают об остановке от арбитра через CMD_QUIT.

3.2. tanks_cond.c: мьютексы, условные переменные и барьер

Модель боя, вывод, журнал, параметры, коды выхода и статистика те же, что в tanks_sem.c; разделы 1–6 файла общие, раздел 7 («Слой синхронизации») написан заново и не содержит sem_t. Участвуют pthread_mutex_t, pthread_cond_t и pthread_barrier_t.

Что на какой фазе и почему.

Фаза	Примитив	Почему
1. Наблюдение, 3. Решение (танки)	Монитор `Dispatch`: `dispatch.lock` + условные переменные `go` и `done`	Состав участников меняется: поток уничтоженного танка завершается. У `pthread_barrier_t` число участников задаётся в `pthread_barrier_init` навсегда, а пересоздать барьер можно, лишь доказав, что на нём никто не стоит, — для этого понадобилась бы ещё одна синхронизация. Условная переменная ждёт предиката над общим состоянием, и число участников — обычный счётчик `pending`, который арбитр задаёт на каждую фазу.
2. Координация	`pthread_barrier_t coord_barrier` на троих (арбитр, координаторы A и B)	Состав не меняется до конца боя, смысл фазы — «все начали, все закончили», это семантика барьера, счётчики и предикаты не нужны. Барьер синхронизирует память (POSIX, 4.12): доски донесений, заполненные до раунда «начало», видны координаторам, `intel[]` — арбитру после раунда «конец».
Доска донесений, приём записей журнала	Мьютексы `board[team].lock`, `journal.lock`	В них одновременно пишут несколько потоков одной фазы.
4–7. Стрельба, уничтожение, перемещение, отрисовка	Нет (пишет один арбитр)	Остальные потоки стоят: танки на `go`, координаторы на барьере. `rwlock` ничего бы не защищал, а ошибку протокола скрыл бы от ThreadSanitizer.
8. Задержка	Нет примитива синхронизации	Обработчик сигнала не может сигналить условную переменную или брать мьютекс (они не async-signal-safe), поэтому досрочный выход из задержки сделан на маске сигналов (`interruptible_delay`).

Диспетчер танков. Арбитр под `dispatch.lock` записывает в ящик `cmd[i]` каждого живого танка команду (`CMD_OBSERVE` или `CMD_DECIDE`), задаёт `pending = N`, делает `cond_broadcast(go)` и ждёт в цикле `while (pending > 0) cond_wait(done)`. Танк в цикле `while (cmd[id] == CMD_NONE) cond_wait(go)` ждёт свою команду (`go` общая для всех, поэтому `broadcast` из-за соседа или ложное пробуждение запустить фазу не могут), забирает команду, обнуляет ящик, выполняет фазу и под тем же мьютексом делает `--pending`; последний отчитавшийся делает `cond_signal(done)`. Переменных две, чтобы отчёт последнего танка будил именно арбитра, а не соседей.

Потерянные пробуждения исключены: команду, `pending` и решение о старте меняют только под мьютексом диспетчера и до `broadcast`/`signal`, а ждущий проверяет предикат под тем же мьютексом в цикле `while`. Не успевший уснуть поток увидит изменение и спать не ляжет. Вложенных блокировок нет: ни один поток не берёт второй мьютекс, удерживая первый, и не встаёт на барьер с захваченным мьютексом.

Фаза координации. Арбитр вызывает `barrier_wait` дважды (`run_coord_phase`): после раунда «начало» координаторы забирают и очищают доски донесений и сводят их в `intel[team]`, до раунда «конец» публикуют результат. Между раундами арбитр ничего не делает, но без него раунд не завершится.

Завершение. Танку арбитр кладёт `cmd[i] = CMD_QUIT` и делает `cond_broadcast(go)` (`retire_tank`). Именно `broadcast`: `signal` мог бы разбудить другой танк, тот снова уснул бы, а адресат не узнал бы о команде, и `join` завис бы. Танк к этому моменту стоит на `go` с пустым ящиком, поэтому команда будет прочитана. Координаторам арбитр ставит флаг `coord_quit` и встаёт на барьер ещё раз: координаторы проходят раунд «начало», видят флаг и выходят, не ожидая раунда «конец». Бой останавливается только между фазами, поэтому раунды строго чередуются, и каждому вызову `barrier_wait` арбитра соответствует ровно один вызов у каждого координатора. Если `pthread_create` не удался на середине, координаторы к барьеру ещё не подходили: до решения `main` (`LAUNCH_GO` или `LAUNCH_ABORT`) они ждут на условной переменной `go`; иначе при неудаче второго первый навсегда остался бы на барьере на троих. Отказ `pthread_create` в программе обработан (код выхода 2).

Сигналы. SIGINT и SIGTERM открыты только в арбитре (и в `main` при настройке); обработчик запоминает сигнал, арбитр проверяет флаг на границе фаз. Ожидания на мьютексе, условной переменной и барьере сигнал не прерывает (POSIX запрещает им возвращать `EINTR`), фаза доводится до конца, как и в `tanks_sem.c`. Ошибка любого примитива приводит к `sync_fatal` (сообщение, восстановление терминала, `abort`).

Сравнение с tanks_sem.c. При тех же аргументах и seed журналы совпадают побайтно, кроме первой строки заголовка (название программы и примитивов) и имени программы в строке «# повтор». Это проверено на 5 seed (1, 2, 3, 42, 2026) и на 300 случайных конфигурациях, на одном процессоре и на нескольких, а также тестом `sem_cond_same_result`.

3.3. Инварианты и детерминизм

Инвариант	**Как обеспечивается**
Две живые машины не занимают одну область	Перемещения применяет один арбитр последовательно; каждый шаг проверяется `move_length`/`can_place` по уже обновлённому полю. После каждого такта вызывается `check_invariants`; нарушение завершает бой с причиной `internal`.
Конфликты движений разрешаются однозначно	Очередь применения фиксируется при расстановке: масса по убыванию, при равной массе — жребий из seed. Опоздавший по очереди проходит только до занятой области; в журнале строка «конфликт клеток … приоритет у …».

Прямой огонь только при наличии линии видимости	Цель выбирается, лишь если первый объект на луче (`cast_ray`) — противник; препятствия и союзники блокируют луч.
Решения принимаются до раскрытия результатов	Фазы 1–3 работают по неизменяемому состоянию мира (арбитр ждёт на `done`); результаты применяются только после получения всех отчётов. Прерывание в фазах 1–3 отбрасывает решения такта целиком.
Отсутствие гонок и взаимоблокировок	Единственный писатель мира — арбитр; мьютексы не вложены. Сборки обеих программ с ThreadSanitizer и ASan+UBSan проходят тесты без отчётов санитайзеров (раздел 6).
Уничтоженный танк удаляется с поля	`apply_deaths` снимает признак `alive`, поток танка получает `CMD_QUIT` (в tanks_cond.c — плюс `broadcast`); в журнале строка об остановке потока.

Детерминизм. Результат не зависит от планировщика: у каждого танка свой `rand_r` (начальное значение `seed + 1 + i`), донесения сводятся коммутативно (`+=`, `|=`), а события журнала перед записью сортируются по ключу (участник, порядковый номер). Проверено: 48 журналов боя с одним seed (20 запусков, 20 запусков на одном ядре через `taskset`, 8 одновременных экземпляров) совпали побайтно (тест `determinism`).

4. Интерфейс

Режимы ввода. Без параметров боя программа запускает интерактивную настройку диалогом ДЗ-1 (ввод через `read(2)`, Enter — значение по умолчанию); любой параметр боя (`--rows`, `--cols`, `--prob`, `--team-a`, `--team-b`) или `--batch`/`--defaults` отключает вопросы. Параметры можно также задать файлом `--config FILE` (строки `key=value`). Ввод проверяется, при ошибке сообщение указывает значение (для конфигурации — имя файла и номер строки).

Отображение. В терминале по умолчанию включена ANSI-перерисовка: поле с танками размером 11 × 5 символов, таблица состояния танков (HP, орудие), строка `Tick: N/лимит | A: .. | B: .. | seed: ..` и последние шесть событий стрельбы и уничтожения; `--no-color` убирает цвета. Если stdout не терминал, или указан `--plain`, вместо перерисовки в stdout построчно выводится полный журнал (наблюдение, донесения и фокус координаторов, решения, выстрелы, попадания, повреждения, конфликты клеток, смены позиций, состояние поля после такта). Тот же журнал всегда пишется в файл.

Задержка и режим без отрисовки. Задержка между тактами — `--delay MS` (0..10000, по умолчанию 350); `--delay 0 --plain` — режим для тестов. Задержка прерывается сигналом.

Фрагмент реального прогона `./tanks_sem --batch --plain --delay 0 --seed 1 --no-color` (всего 7180 строк, фрагменты обрезаны):

```
# tanks_sem — имитация танкового боя (ИДЗ-2, вариант 24), синхронизация: POSIX-семафоры
+ мьютексы
# seed=1 rows=10 cols=10 prob=0.4 max_ticks=600 delay_ms=0 focus=on
```

```
# команда A: случайные типы, 4 танк(ов): A1 fv215b183 A2 leopard_1 A3 mouse A4 leopard_1
# команда B: случайные типы, 4 танк(ов): B1 fv215b183 B2 leopard_1 B3 t62a B4 leopard_1
# препятствия [строка,столбец]: [2,6] [5,4] [8,9]
# потоки: танков 8, координаторов 2, арбитр 1
[T0] начальная расстановка: в строю A 4/4, B 4/4
=== Такт 1 ===
[T1] наблюдение: A3 mouse @(44,0) смотрит вниз | вверх: край поля (3) | вниз:
препятствие (23) | ...
...
[T3] координатор A: донесений 4; противник не обнаружен, фокус огня не назначен
[T3] решение: A3 mouse — противник не виден — намерение: движение вниз на 3
[T3] смена позиции: A3 mouse (44,6) -> (44,9) вниз на 3
...
[T4] решение: B4 leopard_1 — цель A2 leopard_1 (вверх, фокус координатора) — намерение:
прицеливание вверх
[T5] решение: B4 leopard_1 — цель A2 leopard_1 (на линии прицела) — намерение: выстрел
вверх
[T5] выстрел: B4 leopard_1 вверх
[T5] попадание: B4 leopard_1 -> A2 leopard_1 (дистанция 23)
[T5] повреждение: A2 leopard_1 -70 HP (180 -> 110)
[T5] конфликт клеток: B3 t62a (масса 37, очередь #8) претендовал на 2 вниз, область
занята B2 leopard_1
  (масса 40, очередь #4) — приоритет у B2 как более тяжёлого; B3 сдвинулся на 0, B2
удерживает область
...
=== ИТОГИ БОЯ ===
Причина завершения: normal — команда B уничтожена
Результат: победа команды A
Сыграно тактов: 203 (лимит: 600); seed: 1
Разрешено конфликтов перемещения: 9
Решений, изменённых фокусом координатора: A 2, B 0
```

Затем идёт таблица по каждому танку (итог, HP, нанесённый и полученный урон, выстрелы, попадания, уничтожения, тактов, путь) и строка итогов по каждой команде. Для этого прогона: победа A, в строю остался A1 (HP 130/130), 203 такта, 9 конфликтов перемещения; команда A: выстрелов 12, попаданий 12, уничтожено противников 4; команда B: выстрелов 10, попаданий 10, уничтожено 3.

5. Запуск и завершение

Сборка (Makefile в корне): `make` собирает обе программы: `gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread tanks_sem.c -o tanks_sem` и то же для `tanks_cond.c` в `tanks_cond`. Сборка проходит без предупреждений. Дополнительно: `make tsan` (ThreadSanitizer), `make asan` (AddressSanitizer + UBSan + LeakSanitizer) — для обеих программ, `make test` (набор для каждой), `make clean`. Аргументы командной строки и коды выхода у `tanks_cond` те же.

Аргументы командной строки (по `--help`):

Аргумент	Назначение	По умолчанию
`--rows R`	строк поля, 3..12	10
`--cols C`	столбцов поля, 1..12	10

`--prob P`	вероятность препятствия в столбце, 0..1	0.4
`--team-a SPEC`	состав A: `"N"` — N танков случайных типов, `"a b c d"` — количества mouse, fv215b183, t62a, leopard_1	4 случайных
`--team-b SPEC`	состав B, формат как у A	4 случайных (на узком поле меньше)
`--config FILE`	файл `key=value` с теми же ключами	—
`--batch`, `--defaults`	не задавать вопросов	—
`--seed N`	зерно генератора, 0..4294967295	текущее время
`--delay MS`	задержка между тактами, 0..10000 мс	350
`--max-ticks N`	лимит тактов, 0 — без лимита	600
`--no-focus`	координаторы не назначают фокус огня	фокус включён
`--log FILE`	файл журнала событий и итогов	`tanks_sem.log` / `tanks_cond.log`
`--plain`	без ANSI: журнал построчно в stdout	авто, если stdout не терминал
`--ansi`	ANSI-перерисовка, даже если stdout не терминал	—
`--no-color`	ANSI-перерисовка без цветов	—
`--test-signal T:PHASE[:INT|TERM]`	тестовый самосигнал на такте T в фазе PHASE (observe,	—

	coordinate, decide, shoot, destroy, move, render, delay)	
`--help`	справка	—

Коды выхода:

Код	Значение
0	бой завершён: уничтожена команда (или обе) либо достигнут лимит тактов; также `--help`
1	неверные параметры (аргументы, конфигурационный файл, состав команд)
2	системная ошибка: журнал не открыт или записан не полностью, не созданы потоки или примитивы, ошибка вывода на экран, нарушение инварианта модели
3	ввод закончился при интерактивной настройке, бой не начат
130	прерван SIGINT (128 + 2)
143	прерван SIGTERM (128 + 15)

Завершение боя: уничтожена одна команда (победа другой) или обе (ничья); достигнут предел тактов (`--max-ticks`; победителя нет, в итогах приведены число танков в строю и суммарное HP); прерывание SIGINT/SIGTERM (особенно при `--max-ticks 0`); отказ вывода (например, `| head`).

Сигналы: обрабатываются SIGINT и SIGTERM (`sigaction`, без `SA_RESTART`); SIGPIPE игнорируется, закрытый stdout даёт ошибку `EPIPE` и штатную остановку с кодом 2. В `tanks_sem` обработчик взводит флаг и делает `sem_post(g_wake)`, что будит арбитра из задержки; в `tanks_cond` обработчик только запоминает сигнал, а выход из задержки обеспечивает маска сигналов (разд. 3.2). Арбитр проверяет флаг на границе фазы: при сигнале в фазах 1–3 решения такта отбрасываются, при сигнале позже такт доводится до конца. Затем арбитр останавливает все потоки (`CMD_QUIT`; в `tanks_cond` — `QUIT` + `broadcast` танкам и `coord_quit` + раунд барьера координаторам), `main` делает `pthread_join` всех созданных потоков, восстанавливает терминал (если был ANSI-режим: сброс цвета и показ курсора), записывает итоговую статистику в журнал и на экран и возвращает 130 или 143. При Ctrl+C во время интерактивной настройки бой не начинается, код тот же. Если сигнал пришёл уже в последнем такте, когда бой фактически окончен, он не меняет результат: код 0 и причина `normal` или `limit` (это зафиксировано тестом `phase_last_tick`).

Лог: файл по `--log` (по умолчанию `tanks_sem.log` или `tanks_cond.log`), открывается `open(O_WRONLY|O_CREAT|O_TRUNC)` и пишется `write(2)`. Содержимое: заголовок (seed, параметры, команда для повтора боя, составы, препятствия, очередь применения перемещений), события каждого такта и состояние поля после него, в конце блок `=== ИТОГИ БОЯ ===`: причина завершения (`normal`, `limit`, `signal`, `io`, `internal`), результат, сыграно тактов, число конфликтов перемещения, число решений, изменённых фокусом, таблица по танкам и итог по командам.

6. Тесты и результаты

Состав. Каталог `tests/`:

- `battle_tests.sh` — набор для одной программы (запуск: `make test`; режим `--quick` для санитайзеров); результаты в `tests/out/<программа>/`. Группы: справка и неверные аргументы;

граничные поля (3 × 1, 12 × 12 с 6 + 6 танками, ручной состав, лимит 1 такт, ничья); детерминизм; фокус координатора и `--no-focus`; интерактивный ввод (умолчания, ручной, случайный, неверный, слишком длинная строка, EOF, обрыв, Ctrl+C); внешние сигналы SIGINT/SIGTERM, `--max-ticks 0`, восстановление ANSI; сигнал в каждой фазе такта (`--test-signal`); конфигурационный файл и его ошибки; ошибки вывода (журнал недоступен, `| head`);

- `run_tests.sh` — расширенный набор с группами `selftest build cli normal determinism sem_cond signals interactive errors bounds sanitizers`, собирает программы в копии исходников во временном каталоге;
- `check_invariants.py`, `check_log.py`, `log_format.json` — проверка формата журнала и инвариантов (нет двух танков в одной области, стрельба только по видимой цели, согласованность статистики);
- `data/` — входные файлы (ввод для диалога, конфигурации, некорректные данные).

Результаты прогонов:

Команда	**Что проверяет**	**Результат**
`make test`	набор `battle_tests.sh` на релизной сборке	85 PASS, 0 FAIL
`make tsan`	сокращённый набор на сборке с ThreadSanitizer (`halt_on_error=1`)	76 PASS, 0 FAIL
`make asan`	сокращённый набор на сборке ASan + UBSan + LeakSanitizer	76 PASS, 0 FAIL
`bash tests/run_tests.sh`	расширенный набор, обе программы, включая сравнение журналов	320 PASS, 0 FAIL, 0 SKIP

Для `tanks_cond` дополнительно подтверждено: журнал совпал с `tanks_sem` на 5 seed и на 300 случайных конфигурациях (одним процессором и несколькими), сигналы обрабатываются так же, сборки с ThreadSanitizer и с ASan+UBSan прошли по 11 сценариев без отчётов, отказ `pthread_create` обработан. Проверка `valgrind`/`helgrind` не проводилась; её заменяют TSan и ASan+UBSan.

Дополнительно проверено тестировщиками: 10 из 10 внесённых мутаций кода пойманы набором; 303 боя прошли без нарушений инвариантов (проверка `check_invariants.py`). Детерминизм: 48 журналов одного seed совпали побайтно (разд. 3); сравнение двух программ — разд. 3.2.

7. Соответствие требованиям

7.1. Минимальные требования

№	**Требование**	**Где выполнено**
1	Отчёт: ФИО, группа, вариант, условие, сценарий, интерфейс, запуск и завершение	Этот документ, разделы 1–5
2	Корректное выполнение по условию	Разд. 6: 85/85 тестов `battle_tests.sh` и 320/320 в `run_tests.sh`, проверка инвариантов в каждом такте и скриптом `check_invariants.py`

3	Язык C, libc, системные вызовы, pthread	`tanks_sem.c`, `tanks_cond.c`, `-std=c11`, флаг `-pthread`
4	Информативный вывод	Разд. 4, фрагмент реального прогона; полный журнал в режиме `--plain` и в файле
5	Завершение при любых данных	Разд. 5; тесты неверных аргументов, граничных полей, EOF, сигналов (разд. 6)
6	Потоками представлены сущности с поведением	Разд. 2: танки, координаторы, арбитр
7	Асинхронное взаимодействие, мьютексы и семафоры	Разд. 3; `tanks_sem.c` (семафоры + мьютексы) и `tanks_cond.c` (мьютексы + условные переменные + барьер), раздел 7 каждого файла «Слой синхронизации»
8	Результаты работы в отчёте	Разд. 4 и 6

7.2. Дополнительные требования

№	Требование	Где выполнено
1	Комментарии в коде	`tanks_sem.c` и `tanks_cond.c`: шапка с описанием потоков, схемы такта и инвариантов, комментарии к слою синхронизации
2	Ввод-вывод через системные вызовы / дескрипторы	`open`, `read`, `write`, `close` с повтором при `EINTR` и частичной записи (`write_all`, `load_config`, `read_raw_line`, журнал)
3	Завершение по окончанию и по прерыванию, сигналы	Разд. 5; SIGINT/SIGTERM, коды 130/143; тесты `sigint`, `sigterm`, `phase_*`
4	Лог с итоговой статистикой	Файл `--log`, блок `=== ИТОГИ БОЯ ===` (разд. 5)
5	Тестовые наборы и скрипты запуска	Разд. 6; `tests/battle_tests.sh`, `tests/run_tests.sh`, `Makefile`
8	Вторая программа на других примитивах	`tanks_cond.c`: вместо семафоров мьютексы, условные переменные и барьер (разд. 3.2); журнал совпадает с `tanks_sem` при одном seed; `make` собирает обе программы
9	Прочие решения	Детерминизм (журналы одного seed совпадают побайтно), прогон под TSan и ASan+UBSan, самосигнал `--test-signal`, конфигурационный файл

8. Ограничения

- Размеры поля ограничены: строк 3..12, столбцов 1..12; танков в команде не более 6.

- Попадание всегда 100%: цель определяется по тем же позициям, что и при решении, поэтому ветка «промах» недостижима (в проверенных 303 боях не встретилась). Параметров точности, дальности и режима обломков нет; уничтоженные танки всегда удаляются с поля.
- В ANSI-режиме на экране видны только выстрелы и уничтожения (последние шесть событий) и состояние танков; наблюдение, решения, фокус и конфликты клеток полностью отражены только в журнале и в режиме `--plain`.
- Сигнал, пришедший в последнем такте, не меняет результат боя (код 0).
- Пока stdout заблокирован (переполненный канал), SIGTERM может не остановить процесс до освобождения вывода.
- Метка «фокус координатора» в журнале ставится при совпадении цели с фокусом, даже если правило ДЗ-1 выбрало бы ту же цель; реальное влияние фокуса отражает счётчик «решений, изменённых фокусом».
- Тестовый самосигнал `--test-signal` встроен в рабочий код программы.
- Зависимость от платформы: POSIX-семафоры, `pthread_barrier_t` (POSIX.1-2008) и `ppoll` (Linux/glibc); сиды в тестах подобраны под `rand_r` из glibc.